



CrossDBT: An LLVM-Based User-Level Dynamic Binary Translation Emulator

Wei Li, Xiaohui Luo, Yiran Zhang, Qingkai Meng, and Fengyuan Ren^(✉)

Beijing National Research Center for Information Science and Technology (BNRist),
Tsinghua University, Beijing, China
renfy@tsinghua.edu.cn

Abstract. Emulation of Instruction Set Architecture (ISA) is necessary for a wide variety of use cases, such as providing the compatibility to execute programs compiled for a different ISA. This issue is usually solved using Dynamic Binary Translation (DBT), where guest machine code is translated to host ISA on runtime and Just-in-time (JIT) compilation is performed to achieve high-performance emulation. QEMU, a famous emulator, is developed to solve this issue, where Tiny Code Generator (TCG) is constructed to translate guest binary code to TCG Intermediate Representation (IR), and then generate target ISA machine code from TCG IR. Due to the limitations of TCG, some extensions, such as HQEMU, use LLVM as the backend to optimize programs and generate high-performance machine code. However, HQEMU is limited by its underlying implementation. That is, HQEMU still translates guest binary code to TCG IR at first. In this paper, we develop a novel, LLVM-based emulator, where guest machine code is directly lifted to LLVM IR to reduce the extra overhead and produce high-quality machine code. We evaluate our DBT emulator using BYTEmark benchmark and demonstrating its ability to outperform the de facto standard QEMU DBT system. The evaluation results confirm that our emulator delivers an average speedup of 3.3x over QEMU across BYTEmark benchmark compiled for x86-64 running on an ARMv8 platform, meanwhile, demonstrate that our user-level DBT emulator can significantly reduce the overhead to run a program on a cross-ISA system.

Keywords: Dynamic binary translation · Optimization · LLVM

1 Introduction

Over the past several decades, many popular ISAs have been created and the CPU diversity in high performance computing has been increasing, for example, PowerPC, x86, ARM and RISC-V. How to run programs compiled for one ISA, especially closed-source programs, on another ISA platform is a technical issue. User-level DBT is widely used to run Linux/BSD programs compiled for one architecture on another target architecture and plays a prominent role in the transition of ISA. Motivated by the need for ISA transition, a great amount

of work focus on constructing a user-level/system-level DBT emulator such as QEMU [3]. Apple develops Rosetta based on QuickTransit for PowerPC to x86 and Rosetta2 for x86-64 to ARM64. Intel also releases IA-32 execution layer to improve the performance of 32-bit applications running on IA-64 systems. The DBT emulator is also convenient for users to emulate some architectures which they cannot access currently, especially as new features come into these architectures which are not widely available for physical use yet, for example, ARM releases an emulator for users to access SVE feature. A DBT system for user-programs like QEMU works as follows: for initialization, the guest code is mapped into the address space of the emulator and the virtual CPU state is initialized. For translation, the guest code is divided and decoded to many instruction basic blocks, lifting to a low-level IR. Code optimization can be applied on IR and we compile IR to host architecture machine code, which will be cached for performing JIT. Due to the limitation of TCG IR, QEMU utilizes few optimizations and lacks the support of modern SIMD instructions (e.g. AVX, AVX2, AVX512), resulting in the inefficiency of the generated code.

LLVM [19] is a compiler framework that can optimize programs during their whole lifetime, including compile-time, link-time and run-time, where many pass managers and JIT compilers are provided for optimization.

In this work, we present CrossDBT, our novel user-level DBT emulator, which splits the whole process into two stages: the offline stage and the online stage to overcome these issues. In the offline stage, architecture instructions are realized by C++ functions and CrossDBT uses Clang to compile C++ functions to LLVM IR functions. In the online stage, CrossDBT analyzes binary code by an interpreter and divides guest instructions into basic blocks. With LLVM IR functions from the offline stage, guest instructions are mapped into LLVM IR. CrossDBT utilizes LLVM as a backend to optimize LLVM IR, generate high-quality host machine code, and run the code by LLVM JIT.

We evaluate CrossDBT using an x86-64 guest model and a 64-bit ARMv8 host model and implement a DBT emulator outperforming QEMU by an average speedup of 3.3x in BYTEmark benchmark.

Our main contributions are the following:

- We propose an efficient LLVM-based user-level DBT framework called CrossDBT, where guest machine code is directly lifted to LLVM IR in order to avoid the extra overhead and loss of local information caused by translating guest machine code to TCG IR at first.
- We design a two-stage process: offline stage and online stage in a DBT to reduce the cost of compilation while maintaining code quality.
- We build an x86-64-to-ARMv8 user-level DBT emulator to demonstrate the performance of CrossDBT. The x86-64-to-ARMv8 CrossDBT emulator is 3.3 times faster than QEMU in BYTEmark benchmark on average.

This paper is structured as follows: Sect. 2 introduces related works. Section 3 illustrates the CrossDBT architecture in details including the two stages. Section 4 presents the performance evaluation. Finally, we summarize our findings in Sect. 5.

2 Related Work

DBT [24] is a compilation technique to support the execution of binary code for a guest ISA on a host ISA. When the guest ISA and the host ISA are the same, we call it as a same-ISA translator. A general purpose of the same-ISA translator is to instrument binary code, which is referred as dynamic binary instrumentation (DBI). When the guest ISA and the host ISA are different, we call it as a cross-ISA emulator. There are two types of the cross-ISA emulator: system-level DBT emulator running operating systems for any machine and user-level DBT emulator running programs on any supported architecture (on the same OS).

Dynamic Binary Instrumentation. Valgrind [22] lifts code with superblock granularity to VEX IR, where optimizations and modifications can be performed. The instrumenter runs in the same address space as the instrumented program. Dynamo [1] is a software dynamic optimization system that is capable of improving the performance of an instruction stream. DynamoRIO [5,6] allows for heavy-weight transformations and represents programs' code in a low-level, architecture-dependent IR with no optimizations applied. Pin [20] allows inserting calls to functions in the instrumenting tool at any point in the program. DBILL [21] first lifts the machine code to TCG, and then uses LLVM as back-end to conduct optimizations. Instrew [14] overcomes the limitations of TCG through directly lifting machine code to LLVM IR overcoming the limitations of TCG IR.

System-Level DBT Emulator. QEMU [3], as a de facto industry standard, is a typical system-level emulator. QEMU is composed of several subsystems: CPU emulator (with software memory management unit), emulated devices, generic devices, machine descriptions, debugger and user interface. For the translation of machine code, QEMU lifts machine code to TCG IR with basic block granularity and then can be compiled to target architecture machine code. For complex instructions like SIMD instruction, QEMU uses helper C function to emulate. Embra [30] is a machine simulator for the processors, caches, and memory systems of uniprocessors and cache-coherent multiprocessors. Transmeta CMS [11] is a system-level implementation of x86 architecture comprising a native VLIW microprocessor with aggressive speculation, hardware support and adaptive recompilation. MagiXen [7] is a prototype implementation of a Xen [2] virtual machine monitor with an integrated binary translation that can run IA-32 virtual machines on Itanium platforms. PQEMU [12], an extension of QEMU, tries to support multi-processor emulations to effectively utilize the underlying parallelism presented by multi-core processors. HyperMAMBO-X64 [10] runs the translator as a user program under the hypervisor and works with complete transparency with regard to the virtualized system. Captive (2016) [27] exploits existing hardware virtualization technology for improving the performance of full-system cross-ISA emulation. Captive [28] performs DBT within the generated JIT compiler in a virtual bare-metal environment provided by a KVM [17]

hypervisor, which enables us to realize the system-related and system-privileged features not accessible to other DBT systems operating as user processes.

User-Level DBT Emulator. QuickTransit[18] allows software compiled for one specific processor and operating system combination to be executed on a different processor and/or operating system architecture without source code or binary changes. Harmonia [23], an ARM-to-IA DBT tool, is based on open-source QEMU utilizing register mapping and condition-code optimization. ISAMAP [26] is a flexible instruction mapping driven by DBT, whose mapping mechanism providing a fast translation between ISAs under an easy-to-use description. Walkabout [9] develops DBT techniques based on retargetability, separation of machine-dependent from machine-independent concerns and JIT compilation. Walkabout is a framework for experimenting with DBT, as well as other techniques such as interpreters, DBI tools and optimization. Yirr-Ma [29] is an extension to a generated Walkabout machine emulator. Yirr-Ma is used to experiment with two different DBT techniques: (1) translates hot source machine traces using dynamic partial inlining of interpretation functions, (2) translates hot source machine traces by JIT compiling an IR of the trace’s dynamic operational semantics. LLVM QEMU [8] addresses the limitation of TCG IR. The guest machine code is first lifted to micro-operations, which are compiled to LLVM IR. Then LLVM QEMU utilizes LLVM backend to optimize IR and generate host machine code. HQEMU [16] solves the performance problem of QEMU by using a hybrid approach where only performance-sensitive sections are optimized using LLVM. LnQ [16] achieves the performance improvement by directly lifting machine code to LLVM IR and applying the chaining optimization such as caches for predicting indirect jump and function returns. Due to the limitations of TCG, HQEMU is not possible to further increase translation granularity to allow more complex control flows and exploits more semantic information for optimization. [13] implements a novel approach that adapts the Instrew/Rellume [14] framework which directly lifts machine code to LLVM IR with function-level granularity.

Summary. Many approaches are based on QEMU’s DBT [8,15,16] and other binary translation systems [4,25] use LLVM as the backend to generate host machine code from scratch. We further discuss these emulators and compare them with our work. LLVM QEMU [8] lifts machine code to micro operations of TCG IR at first. Each micro operation is implemented by C and compiled into LLVM IR, which is used to generate host machine code by LLVM backend. HQEMU [15] uses a hybrid approach where the entire code is also first translated using TCG and only performance-sensitive sections are then optimized using LLVM. LnQ [16] achieves performance improvements over QEMU by lifting guest code to LLVM IR. But they still have some limitations. As the machine code is lifted first to TCG IR, LLVM QEMU and HQEMU suffer from two problems: (1) TCG IR loses the local context information of the original code, which can be used for LLVM to optimize code, (2) The performance improvements

are provided by LLVM and using TCG IR only costs more time with no benefit. LnQ, which is an extension of QEMU, faces another problem that QEMU focuses more on compatibility not performance and then doesn't utilize many specific optimizations for user-level DBT emulator. Table 1 describes the differences between CrossDBT and related work, clearly shows the novel features of CrossDBT.

Table 1. The comparison between CrossDBT and related works

Tool	Frontend	Backend	Translation Path	Main Contributions
QEMU	TCG	TCG	Guest code→TCG IR→ Host code	System-level emulation
LLVM QEMU	TCG	LLVM	Guest code→TCG IR→ LLVM IR→Host code	Lifting TCG IR to LLVM IR
HQEMU	TCG	LLVM	Guest code→TCG IR→ LLVM IR→Host code	Optimizing hot-spot sections
LnQ	TCG	LLVM	Guest code→LLVM IR→ Host code	Lifting binary code to LLVM IR
CrossDBT	Interpreter	LLVM	Guest code→LLVM IR→ Host code	Two-stage translation

3 DBT Emulator

3.1 Overview

In this section, we describe the main concepts of CrossDBT, which are composed of two main stages as shown in Fig. 1: offline stage and online stage.

In offline stage, we implement the instructions of the target architecture by C++ functions and then compile them to LLVM IR, which is discussed in Sect. 3.2. The instructions are described in LLVM IR files for mapping in the runtime. The online stage is discussed in Sect. 3.3. The DBT system itself runs in host system as a process holding the heap and stack memory for the guest application. The purpose to separate the DBT system into the offline stage and online stage is for hiding the overhead of translating C++ functions to LLVM IR and utilizing optimization. In online stage, CrossDBT only needs to load the LLVM IR files produced on the offline stage. We use an x86-64-to-ARMv8 user-level emulator as an example to explain our CrossDBT framework in this section. It is not difficult to port it to a new ISA. What we should do is to emulate the instructions according to the manual of the new ISA without the support of the vendors.

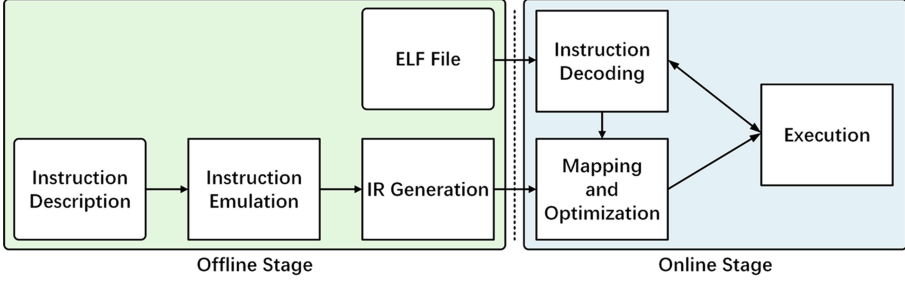


Fig. 1. Overview of CrossDBT.

3.2 Offline Stage

ISA Description. To support a new ISA in CrossDBT, we first have to acknowledge the ISA description according to the ISA developer’s manual such as Intel 64 and IA-32 architectures software developer’s manual for x86-64. A classical ISA includes these features:

- instruction types: data handling and memory operations, arithmetic and logic operations, control flow operations, coprocessor instructions, complex instructions (e.g. floating-point arithmetic operations, atomic instructions and single instruction multiple data (SIMD) instructions).
- instruction encoding: opcode formats, number and type of operands, immediate value formats, operand addressing.
- instruction length: fixed length, variable length.
- register pressure: the availability of free registers at any point in time during the program execution.
- register type: general-purpose registers, special registers (e.g. segment registers and EFLAGS registers in x86-64).

We should consider these features when designing the vCPU structure, which includes general-purpose registers, extend control registers, system registers, model-specific registers, SIMD registers and x87 registers for x86-64 ISA.

Instruction Emulation. We use the high-level programming language C++ to define instruction syntax and semantics. The instruction function is identified by opcode, operand type and size. For simple instructions, we implement them as shown in Fig. 2. For complex instructions, we design specific approach for each type of complex instruction. For example, we utilize the interface provided by LLVM to construct our atomic instructions. LLVM provides a uniformed interface to realize atomic instructions for different ISA such as `__atomic_fetch_add` and `__atomic_add_fetch`. SIMD and floating-point instructions, system calls are discussed in Sect. 3.4 and Sect. 3.5.

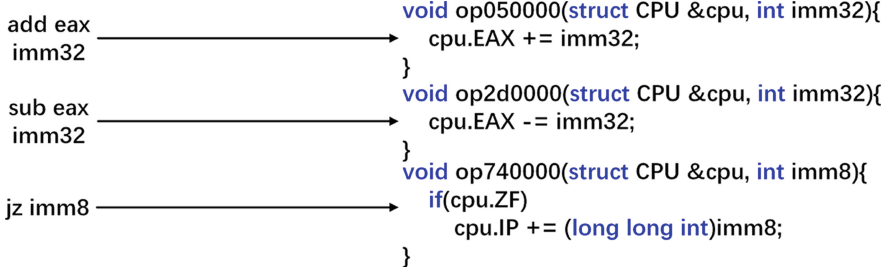


Fig. 2. The C++ functions for emulating instructions.

```

void op2d0000(struct CPU &cpu, int imm32) {
    cpu.EAX -= imm32;
}
↓
define dso_local void @_Z4op2d0000R3CPUi
(%struct.CPU* nonnull align 8 dereferenceable(16) %0, i32 %1) #0 {
    %3 = alloca %struct.CPU*, align 8;          %4 = alloca i32, align 4;
    store %struct.CPU* %0, %struct.CPU** %3, align 8;
    store i32 %1, i32* %4, align 4;              %5 = load i32, i32* %4, align 4;
    %6 = load %struct.CPU*, %struct.CPU** %3, align 8;
    %7 = getelementptr inbounds %struct.CPU, %struct.CPU* %6, i32 0, i32 1
    %8 = load i32, i32* %7, align 8;              %9 = sub nsw i32 %8, %5;
    store i32 %9, i32* %4, align 4;              ret void;
}

```

Fig. 3. The LLVM IR function compiled from the C++ function.

IR Generation. In order to get LLVM IR functions, we use Clang to compile these instruction C++ functions to LLVM IR functions stored in LLVM IR files. The LLVM IR functions are named, which can be loaded by name in online stage. Figure 3 shows that a C++ function is compiled to a corresponding LLVM IR function. Due to the implementation of C++ static polymorphism, the name of the LLVM IR function is not the same as the name of C++ function. Thus, an instruction description table is needed, where CrossDBT can get the LLVM IR function of any instruction.

3.3 Online Stage

The online stage of CrossDBT involves the analysis and running of the guest application and is composed of three main phases: Instruction Decoding (ELF Loader, Dispatch Engine, Interpreter), Mapping and Optimization, Execution as shown in Fig. 4.

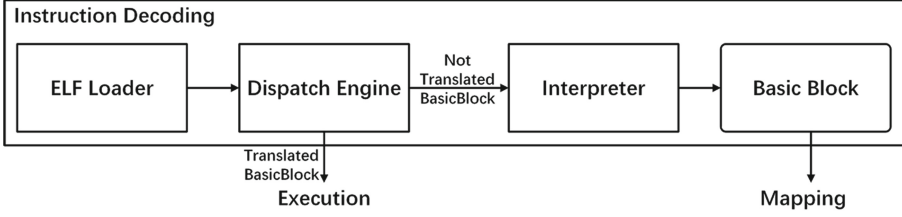


Fig. 4. The architecture of instruction decoding phase.

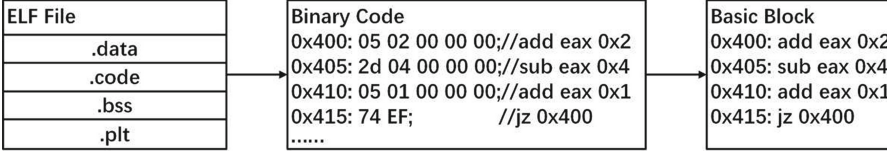


Fig. 5. Instruction decoding phase analyzes ELF files and decodes instructions.

Instruction Decoding. The first phase in online stage is instruction decoding, which is composed of ELF Loader, Dispatch Engine, Interpreter as shown in Fig. 4. ELF Loader loads ELF files and extracts useful information like code segment, data segment, the relocation table. After loading files, CrossDBT uses dispatch engine to check whether this basic block of instructions has been cached, where basic block is identified by the Program Counter (PC) of the first instruction. If the basic block is cached, the next phase is the execution phase. If not, CrossDBT decodes instruction by the interpreter and the next phase is mapping phase. The interpreter analyzes binary code according to ISA developers' manual. As a user-level emulator, CrossDBT only focuses on the emulation of normal operations and ignores the privilege instructions, I/O operations. Figure 5 gives an example to show how instruction decoding works.

Mapping and Optimization. During the mapping and optimization phase, CrossDBT creates an empty LLVM IR function representing a basic block. For each decoded function in the basic block, CrossDBT puts the corresponding LLVM IR function (which was produced in the offline stage) in it. We should optimize the LLVM IR function to generate high performance host machine code. LLVM provides various kinds of LLVM IR pass manager for optimization such as function inlining pass, instruction combining pass, loop vectorize pass. In Fig. 6, we give an example to show the pipeline mapping the instructions in a basic block to LLVM IR functions and optimizing LLVM IR functions by LLVM pass managers.

Execution. There are two situations that CrossDBT gets into the execution phase. The first situation is that CrossDBT tries to generate host machine code

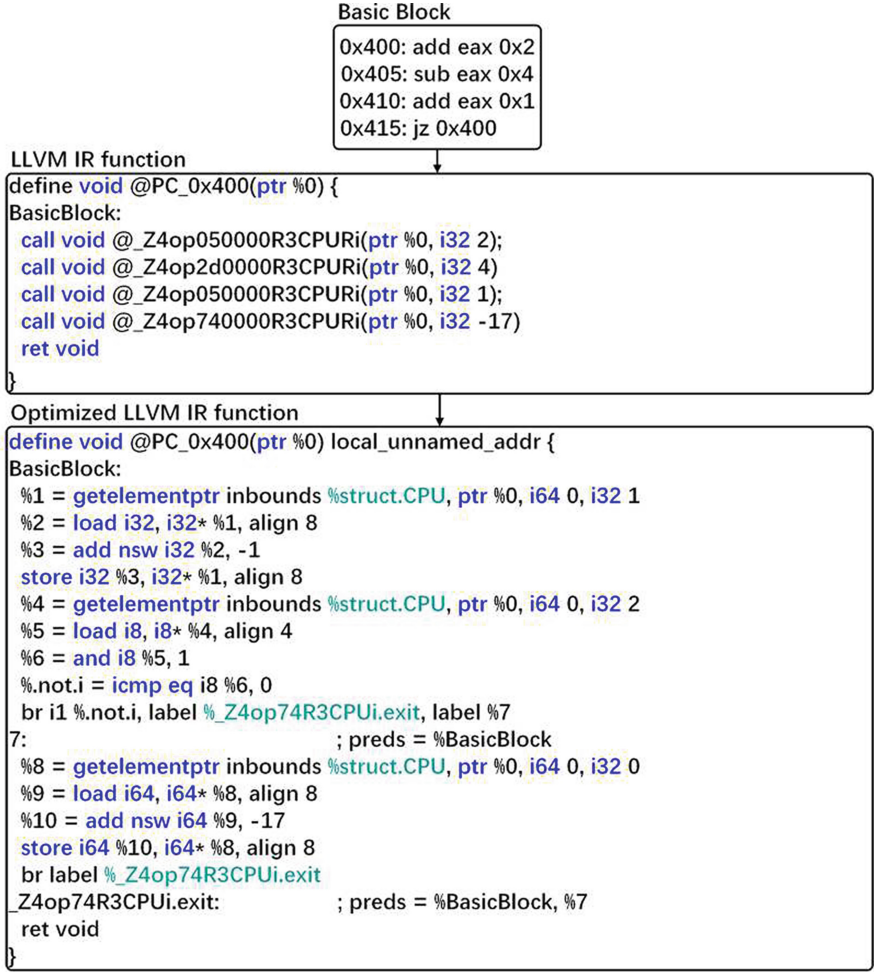


Fig. 6. Mapping and optimization phase generates LLVM IR from instructions.

.text	.globl PC_0x400	.p2align 2
.type PC_0x400,@function	.globl PC_0x400	
PC_0x400:		
.cfi_startproc	ldr w8, [x0, #8]	ldrb w9, [x0, #12]
sub w8, w8, #1	str w8, [x0, #8]	tbz w9, #0, .LBB0_2
ldr x8, [x0]	sub x8, x8, #17	str x8, [x0]
.LBB0_2:		
ret		
.Lfunc_end0:		
.size PC_0x400,	.Lfunc_end0-PC_0x400	.cfi_endproc

Fig. 7. The generated host machine code from LLVM IR in Fig. 6.

by LLVM backend after finishing mapping phase. CrossDBT passes the LLVM IR function to LLVM JIT for code generation and code cache, notifies dispatch engine that this basic block has been cached now. The next time coping with this basic block, CrossDBT does not need to translate it again. The second case is that CrossDBT tries to run the host machine code by the LLVM JIT for the basic block cached. For this case, we don't have to utilize extra operations. Figure 7 shows an example of the generated host machine code.

LLVM has several JIT methods for users: JIT, MCJIT, ORCJIT. ORCJIT is the newest JIT method provided by LLVM, which has many new technical features such as JIT-linking, LLVM IR compilation, eager and lazy compilation, concurrent JIT'd code and concurrent compilation. These features provide more flexibility to organize the ORC layers to achieve better performance. Thus, the JIT in CrossDBT is built by LLVM ORCJIT.

3.4 SIMD and Floating Point

SIMD instructions are widely used to process 3D graphics, although modern graphics cards with embedded SIMD have largely taken over this task from the CPU. Some systems also include permute functions that re-pack elements inside vectors, making them particularly useful for data processing and compression. They are also used in cryptography. The trend of general-purpose computing on GPUs may lead to wider use of SIMD in the future. Even some glibc functions (e.g. memcpy) use SSE instructions as default in x86-64 architecture. QEMU 6.0.0 supports MMX, SSE, SSE2, SSE3, SSSE3, SSE4 instruction sets by using helper functions to emulate. Due to the limitation of helper functions, QEMU cannot utilize the hardware to accelerate SIMD instruction. CrossDBT also implements MMX, SSE, SSE2, SSE3, SSSE3, SSE4 instructions by emulation. However, with the optimization of LLVM backend, CrossDBT makes use of the hardware for SIMD instruction to a certain degree.

x87 is a floating-point-related subset of x86 ISA. It originated as an extension of the 8086 instruction set in the form of optional floating-point coprocessors that worked in tandem with corresponding x86 CPUs. Most x86 processors since the Intel 80486 have had these x87 instructions implemented in the main CPU, but the term is sometimes still used due to the feature of providing 80-bit floating-point number or the compatibility for 32-bit application. Modern processors have SSE/AVX instruction set extension providing floating-point instructions which operate on a new independent registers, the XMM and YMM registers. CrossDBT supports the floating-point operations supplied by x87 and SSE instructions. There are notable differences in the method that floating-point flags, NaNs, rounding modes and infinities are handled in specific architecture. This incompatibility between floating point needs to be considered in some cases. CrossDBT ensures that the guest vCPU state is accurate with how the guest machine would normally operate. For example, a precision mismatch is handled by utilizing software floating-point library such as 128-bit floating-point number provided by compiler. Another example is about rounding modes that Fused Multiply Add (FMA) instructions calculate $a * b + c$ with only once intermediate

value rounding while using normal arithmetic operations would operate twice intermediate value rounding.

3.5 System Call

A system call is the programmatic way in which a computer program requests a service from the kernel of the operating system on which it is executed. System calls provide an essential interface between a process and the operating system. Implementing system calls requires a transfer of control from user space to kernel space, which involves some sort of architecture-specific feature. Thus, system calls are different for x86-64 and ARMv8 architecture. CrossDBT needs an immediate layer to translate system call and only deals with the situation that guest and host both use the Linux OS. There are three cases about how to translate a system call.

Directly Use Host System Call. The most part of system calls have the same interface and the same semantics such as read and write. CrossDBT can get the arguments from the guest registers and directly invoke the host system call to achieve the goal.

Emulate by Host System Call. For some system calls, they have different interface between x86-64 and ARMv8 such as stat, for which we should prepare the arguments to call the host system call. For system calls having special features such as clone, we need to pay more effort. CrossDBT uses fork to create a new process and sets thread local storage, child tid, parent tid, child stack of the child process. For creating a new thread, CrossDBT utilizes pthread libs to construct a new thread and sets the necessary data of the thread, where a new translation main loop is established to continue the translation. When the task of thread or process is finished, CrossDBT uses exit syscall to close and sets child_tidptr to zero if needed.

Linux vDSO and Vsyscall. The vsyscall and vDSO (virtual Dynamic Shared Object) are two mechanisms used to accelerate certain system calls in Linux. The vsyscall is added as a way to execute specific system calls which do not need any real level of privilege to run in order to reduce the system call overhead. The kernel maps into user space a page containing the fast implementation of some system calls. However, this vsyscall mechanism has some limitations: the memory allocated is small and allows only 4 system calls, and, more important and serious, the vsyscall page is statically allocated to the same address in each process, which causes a security problem. Then the vDSO offers the same functionality as the vsyscall, while overcoming its limitations. The vDSO is a memory area allocated in user space which exposes some kernel functionalities at user space in a safe manner. For Linux kernel, if vDSO is not supported, kernel falls back to the traditional system calls, which means

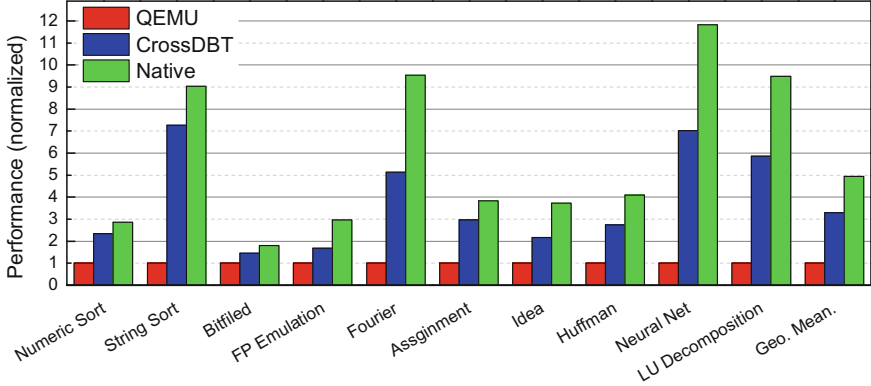


Fig. 8. Performance of QEMU, CrossDBT and Native on BYTEmark (higher is better).

we don’t need to do anything. For glibc functions like `gettimeofday`, if `vDSO` is not supported, these functions try to use `vsyscall` in `x86-64`, which we need to cope with. The `vsyscall` always maps system calls into process space from `0xFFFFFFFF600000` to `0xFFFFFFFF600C00`, where the `gettimeofday`, `time`, `getcpu` system calls locate. They are emulated by the corresponding host system calls in CrossDBT.

4 Experiment Results

To evaluate the performance of CrossDBT, we apply it on the BYTEmark benchmark, which is designed to expose the capabilities of a system’s CPU, FPU, and memory system and also used to for the evaluation of similar systems [3]. We use `x86-64` as guest architecture and `ARMv8` as host architecture. The performance of CrossDBT is compared against a native compilation for `ARMv8` as well as against QEMU 4.2.0.

4.1 Setup

We conduct the following experiments in a single thread mode. Our `ARMv8` target platform is based on Kunpeng 920-4826 CPUs, 64 KB L1 instruction cache, 64 KB L1 data cache, 512 KB L2 cache, 48 MB L3 cache, 2.6 GHz frequency and 192 GB memory, running Ubuntu Linux 18.04 with Linux Kernel 5.0.0-23. The native code is compiled with Clang+LLVM 11.0.0 and `-O3`.

4.2 Results

We first compare the running performance of QEMU, CrossDBT and native ARMv8. The result is shown in Fig. 8. The performance ranges from 1.46X to 7.26X, and CrossDBT is 3.3 times faster than QEMU on a geometric average of BYTEmark benchmark.

On the one hand, the improvement is significant in benchmarks that have heavy floating-point operations such as String Sort, Fourier, Neural Net and LU Decomposition. On the other hand, CrossDBT doesn't reach high improvement on the benchmarks that have few floating-point operations.

The main reason of this improvement is the insufficient translation ability of QEMU. TCG, the compiler of QEMU's DBT, doesn't support floating-point operations or SIMD instructions and translates them into helper function calls, which causes huge performance loss. Thus, CrossDBT gains much performance improvement than QEMU on floating-point-intensive benchmark because CrossDBT utilizes LLVM backend to generate high quality code and make use of the host hardware to accelerate SIMD instructions and floating-point operations.

Comparing CrossDBT with the native ARMv8, the extra overhead ranges 18.2% to 46.1% and 33.1% on geometric average. Coincidentally, the largest gap is also happening on floating-point-intensive benchmark. This phenomenon indicates that we need pay more effort on how to translate floating-point operations and SIMD instructions better.

5 Conclusion

In this article, we developed a novel user-level DBT emulator, CrossDBT, that directly lifts guest machine code to LLVM IR and utilizes LLVM backend to optimize and generate target host machine code. CrossDBT is composed of two stages: offline stage, where we implement guest instructions by C++ functions and compile them into LLVM IR, and online stage, where we lift binary code to LLVM IR and generate host machine code.

We demonstrate the performance of CrossDBT by using an x86-64 guest running normal environment on ARMv8 host, where CrossDBT outperforms the widely used QEMU across BYTEmark benchmark with on average 3.3 times.

Acknowledgments. The authors gratefully acknowledge the anonymous reviewers for their constructive comments. This work is supported in part by the National Key Research and Development Program of China (No.2018YFB1700203), and by National Natural Science Foundation of China (NSFC) under Grant No. 62132007 as well as gifts from Huawei.

References

1. Bala, V., Duesterwald, E., Banerjia, S.: Dynamo: a transparent dynamic optimization system. SIGPLAN Not. **35**(5), 1–12 (2000). <https://doi.org/10.1145/358438.349303>

2. Barham, P., et al.: Xen and the art of virtualization. In: Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles, SOSP 2003, pp. 164–177. Association for Computing Machinery, New York (2003). <https://doi.org/10.1145/945445.945462>
3. Bellard, F.: QEMU, a fast and portable dynamic translator. In: 2005 USENIX Annual Technical Conference (USENIX ATC 2005), Anaheim, CA. USENIX Association, April 2005
4. Brandner, F., Fellnhöfer, A., Krall, A., Riegler, D.: Fast and accurate simulation using the LLVM compiler framework. In: Proceedings of the 1st Workshop on Rapid Simulation and Performance Evaluation: Methods and Tools, RAPIDO, vol. 9, pp. 1–6 (2009)
5. Bruening, D., Amarasinghe, S.: Efficient, transparent, and comprehensive runtime code manipulation. Ph.D. thesis, Massachusetts Institute of Technology, Department of Electrical Engineering and Computer Science (2004)
6. Bruening, D., Garnett, T., Amarasinghe, S.: An infrastructure for adaptive dynamic optimization. In: Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization, CGO 2003, pp. 265–275. IEEE Computer Society, USA (2003). <https://doi.org/10.5555/776261.776290>
7. Chapman, M., Magenheimer, D.J., Ranganathan, P.: Magixen: combining binary translation and virtualization. HP Enterprise Systems and Software Laboratory, pp. 1–15 (2007)
8. Chipounov, V., Candea, G.: Dynamically translating x86 to LLVM using QEMU. Technical report, EPFL (2010)
9. Cifuentes, C., Lewis, B., Ung, D.: Walkabout-a retargetable dynamic binary translation framework. In: Workshop on Binary Translation, pp. 22–25 (2002)
10. d’Antras, A., Gorgovan, C., Garside, J., Goodacre, J., Luján, M.: Hypermambo-x64: using virtualization to support high-performance transparent binary translation. In: Proceedings of the 13th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, VEE 2017, pp. 228–241. Association for Computing Machinery, New York (2017). <https://doi.org/10.1145/3050748.3050756>
11. Dehnert, J.C., et al.: The transmeta code morphingTM software: using speculation, recovery, and adaptive retranslation to address real-life challenges. In: Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization, CGO 2003, pp. 15–24. IEEE Computer Society, USA (2003). <https://doi.org/10.5555/776261.776263>
12. Ding, J.H., Chang, P.C., Hsu, W.C., Chung, Y.C.: PQEMU: a parallel system emulator based on QEMU. In: 2011 IEEE 17th International Conference on Parallel and Distributed Systems, pp. 276–283 (2011). <https://doi.org/10.1109/ICPADS.2011.102>
13. Engelke, A., Okwieka, D., Schulz, M.: Efficient LLVM-based dynamic binary translation. In: Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, VEE 2021, pp. 165–171. Association for Computing Machinery, New York (2021). <https://doi.org/10.1145/3453933.3454022>
14. Engelke, A., Schulz, M.: Instrew: leveraging LLVM for high performance dynamic binary instrumentation. In: Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, VEE 2020, pp. 172–184. Association for Computing Machinery, New York (2020). <https://doi.org/10.1145/3381052.3381319>

15. Hong, D.Y., et al.: HQEMU: a multi-threaded and retargetable dynamic binary translator on multicores. In: Proceedings of the Tenth International Symposium on Code Generation and Optimization, CGO 2012, pp. 104–113. Association for Computing Machinery, New York (2012). <https://doi.org/10.1145/2259016.2259030>
16. Hsu, C.C., et al.: LNQ: building high performance dynamic binary translators with existing compiler backends. In: 2011 International Conference on Parallel Processing, pp. 226–234 (2011). <https://doi.org/10.1109/ICPP.2011.57>
17. Kivity, A., Kamay, Y., Laor, D., Lublin, U., Liguori, A.: KVM: the Linux virtual machine monitor. In: Proceedings of the Linux Symposium, Dttawa, Dntorio, Canada, vol. 1, pp. 225–230 (2007)
18. Knowles, P.: Transitive and quicktransit overview (2008). <https://www.linux-kvm.org/images/9/98/KvmForum2008%24kdf2008.2.pdf>
19. Lattner, C., Adve, V.: LLVM: a compilation framework for lifelong program analysis and transformation. In: Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization, CGO 2004, p. 75. IEEE Computer Society, USA (2004). <https://doi.org/10.5555/977395.977673>
20. Luk, C.K., et al.: Pin: building customized program analysis tools with dynamic instrumentation. In: Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2005, pp. 190–200. Association for Computing Machinery, New York (2005). <https://doi.org/10.1145/1065010.1065034>
21. Lyu, Y.H., et al.: DBILL: an efficient and retargetable dynamic binary instrumentation framework using LLVM backend. In: Proceedings of the 10th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, VEE 2014, pp. 141–152. Association for Computing Machinery, New York (2014). <https://doi.org/10.1145/2576195.2576213>
22. Nethercote, N., Seward, J.: Valgrind: a framework for heavyweight dynamic binary instrumentation. In: Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2007, pp. 89–100. Association for Computing Machinery, New York (2007). <https://doi.org/10.1145/1250734.1250746>
23. Ottoni, G., Hartin, T., Weaver, C., Brandt, J., Kuttanna, B., Wang, H.: Harmonia: a transparent, efficient, and harmonious dynamic binary translator targeting the intel® architecture. In: Proceedings of the 8th ACM International Conference on Computing Frontiers, CF 2011. Association for Computing Machinery, New York (2011). <https://doi.org/10.1145/2016604.2016635>
24. Probst, M.: Dynamic binary translation. In: UKUUG Linux Developer’s Conference, vol. 2002 (2002)
25. Shigenobu, K., Ootsu, K., Ohkawa, T., Yokota, T.: A translation method of arm machine code to LLVM-IR for binary code parallelization and optimization. In: 2017 Fifth International Symposium on Computing and Networking (CANDAR), pp. 575–579 (2017). <https://doi.org/10.1109/CANDAR.2017.75>
26. Souza, M., Nicácio, D., Araújo, G.: ISAMAP: instruction mapping driven by dynamic binary translation. In: Varbanescu, A.L., Molnos, A., van Nieuwpoort, R. (eds.) ISCA 2010. LNCS, vol. 6161, pp. 117–138. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-24322-6_11
27. Spink, T., Wagstaff, H., Franke, B.: Hardware-accelerated cross-architecture full-system virtualization. *ACM Trans. Archit. Code Optim.* **13**(4), 1–25 (2016). <https://doi.org/10.1145/2996798>

28. Spink, T., Wagstaff, H., Franke, B.: A retargetable system-level DBT hypervisor. *ACM Trans. Comput. Syst.* **36**(4), 1–24 (2020). <https://doi.org/10.1145/3386161>
29. Tröger, J.: Specification-driven dynamic binary translation. Ph.D. thesis, Queensland University of Technology (2005)
30. Witchel, E., Rosenblum, M.: Embra: fast and flexible machine simulation. In: *Proceedings of the 1996 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS 1996*, pp. 68–79. Association for Computing Machinery, New York (1996). <https://doi.org/10.1145/233013.233025>